

MongoDB: Um Gerenciador de Banco de Dados NoSQL



Conteudista: Prof. Dr. Cleber Silva Ferreira da Luz

Objetivos da Unidade:

- Compreender a ferramenta *MongoDB*;
- Estudar o conceito do NoSQL;
- Conhecer o conceito de manipulação de dados.



 Material Teórico



 Material Complementar



 Referências



Material Teórico

Introdução

Bancos de dados são poderosas ferramentas para manipular dados, e possuem, o objetivo de separar os dados das aplicações. Possuem também, o objetivo de armazenar os dados permanente. A criação de um banco de dados, se dá através de *softwares* específicos, chamados de “**Sistemas Gerenciadores de Banco de Dados**”, ou são chamados simplesmente de **SGBD**. Para Silberschatz (2020, p. 1) SGBDs são formados por coleção de dados inter-relacionados, e geralmente, possuem um conjunto de comandos que permitem manipular os dados de forma eficiente. Assim, um SGBD visa fornecer uma forma de armazenamento e recuperação de dados eficiente.

Atualmente, há vários SGBDs disponíveis no mercado. Eles se diferem um do outro pela forma como os dados são armazenados e manipulados. Por exemplo, SGBDs relacionais manipulam dados utilizando técnicas de relacionamentos de dados, algumas técnicas são chaves primárias, chave estrangeiras e entre outros. Já os SGBDs não relacionais, que é o foco desta Unidade,

armazenada dados utilizando técnicas de manipulação de dados em grandes escalas, e que os dados não possuem uma relação diretamente entre si. Ainda existem outros tipos de SGBDs, tais como; orientados a objetos, hierárquico, redes e entre outros.

Mesmo com tantas opções de SGBDs é fundamental a escolha de um SGBD eficiente e de acordo com o modelo de banco de dados que você deseja implementar. Por exemplo, para aplicações de *Big Data*, é mais comum o uso de SGBDs não relacionais, pois estes tipos de SGBDs trabalham muito bem com grande quantidade de dados.

Banco de dados não relacionais são chamados de NoSQL, isso porque os bancos de dados relacionais, implementam a linguagem SQL. Dessa forma, NoSQL, é uma forma de se dizer, não relacional. O foco desta Unidade de ensino é explanar todos os conceitos por trás dos bancos NoSQL e realizar uma pratica com um SGBD não relacional. Quando você ouvir falar em banco de dados não relacional, você poderá abstrair que está se falando de bancos NoSQL, ou vice e versa.

NoSQL

Vivemos na era da informação digital, onde, são geradores milhões de dados por segundo. Essas informações digitais são extraídas através de grandes quantidades de dados. Assim, o armazenamento e o processamento massivo de dados permitem

extrair informações que muitas vezes, podem ser valiosas para muitas empresas. Dando início assim, a era do *Big Data*.

Com essa nova era de informação digital, surgiram novas tecnologias e ferramentas que possibilitam o armazenamento e o processamento de tais dados. No seguimento de banco de dados, surgiu o NoSQL. Bancos NoSQL são banco de dados robustos e muito eficientes, banco de dados que são projetados para manipular dados em grande escala.

De fato, o NoSQL é uma tecnologia que surgiu através do *Big Data*. Habitualmente, banco de dados NoSQL são implementados em grandes computadores, chamados de *Clusters*, que consistem em um supercomputador formado por vários computadores trabalhando em conjunto através de uma rede de computadores. Com a utilização de um *Cluster* a aplicação se torna mais robusta, por exemplo, aplicações NoSQL são totalmente **tolerantes a falhas**, pois, se um computador falhar, os demais computadores continuam trabalhando normalmente. A seguir, vamos expor mais algumas vantagens do NoSQL.

Outra vantagem do NoSQL consiste no fácil **poder de escalar a aplicação**. Por exemplo, se a quantidade de dados aumentar inesperadamente, é possível adicionar mais um computador de forma rápida e simples, para que seja possível processar toda a quantidade de dados daquele momento. O processo inverso, também é possível, se a quantidade de dados diminuir, é possível

diminuir a quantidade de máquinas no *Cluster*. Essa questão de aumentar ou diminuir as máquinas no Cluster é chamada de escalabilidade horizontal.

Ainda sobre as vantagens do NoSQL podemos falar sobre a rapidez no processamento de dados. Para aplicações que manipulam grande quantidade de dados, NoSQL possui uma performance muito valiosa, permitindo manipular uma quantidade massiva de dados de forma rápida e eficiente.

Para finalizar, podemos citar ainda, a flexibilidade do NoSQL em manipular os dados. No NoSQL não há algum esquema pré-definido, ou ainda, os dados não possuem formato e tamanho específicos. Assim, o NoSQL possui a capacidade de manipular dados em qualquer formato.

NoSQL x SQL

Como mencionado na seção anterior, banco de dados não relacionais são chamados de NoSQL. Enquanto banco de dados relacionados são chamados de SQL, isto é, porque banco de dados relacionais utilizam a linguagem SQL. Dessa forma, já podemos observar a principal diferencia entre esses bancos, que consiste na forma como os dados são organizados. Vamos falar rapidamente sobre banco de dados relacionais.

Banco de dados relacionais se baseiam na filosofia que os dados possuem um relacionamento entre si. Esta é a principal característica de banco de dados relacionais. Banco de dados relacionais utilizam a linguagem SQL, que consiste em uma linguagem criada em meados dos anos 70, para padronizar a manipulação de dados em banco de dados relacionais.

Habitualmente, banco de dados relacionais, possuem esquemas de armazenamento de dados. Possuem dados em formato e tamanho específicos. Os dados são estruturados, isto é, dados que ao serem armazenados, ou recuperados, são armazenados em estruturas, chamadas tabelas. Uma tabela consiste em um conjunto de linhas e colunas. As colunas são as propriedades, ou informações que desejamos armazenar e cada linha representa uma instância desse conjunto de dados.

Sobre um último comentário sobre banco de dados relacionais, podemos citar que banco de dados relacionais utilizam o conceito de chaves para realizar o relacionamento entre os dados. São utilizados os conceitos de chave primária e chave estrangeiras para realizar o relacionamento de dados.

Agora, sobre o NoSQL, podemos começar a falar sobre NoSQL dizendo que o NoSQL é tudo ao contrário do que foi exposto acima. Entretanto, vamos deixar claro as características do NoSQL, para fixar bem a diferença entre NoSQL e SQL.

Vamos começar falando que o NoSQL é um tipo de banco de dados que utiliza a filosofia de dados não relacionais. Isto é, para o NoSQL os dados não possuem uma relação indiretamente ou diretamente. O NoSQL trabalha com dados não estruturados. Dados que não possuem um formato específico ou trabalho específico. Os dados são armazenados em estruturas, geralmente chamadas de coleções.

Atualmente, o banco de dados *Mongodb* é uma boa opção para trabalhar com banco de dados não estruturados. *MongoDB* é um sistema de gerenciamento de banco de dados NoSQL, criado em 2007 pela empresa *MongoDB Inc.* Ele difere de bancos de dados relacionais tradicionais, pois não usa tabelas, linhas e colunas, mas sim documentos e coleções.

Um documento no *MongoDB* é uma estrutura de dados semelhante a um *JSON (JavaScript Object Notation)*, que pode conter campos e valores. Esses documentos são armazenados em coleções, que podem ser comparadas a tabelas em um banco de dados relacional.

MongoDB

O *MongoDB* é uma opção poderosa e flexível para empresas e desenvolvedores que precisam lidar com grandes quantidades de dados e querem aproveitar as vantagens do modelo NoSQL. Com

sua escalabilidade horizontal e documentação fácil de usar, ele é uma escolha popular para muitos projetos de *software* modernos.

Instalação do *MongoDB*

Como exposto anteriormente, atualmente, estão dispostos no mercado diversos SGBDs NoSQL. Todos trabalham muito bem com dados em grande escala e dados não estruturados. Aqui, iremos adotar o *MongoDB* para o SGBD principal para praticamos os conceitos de bancos de dados NoSQL.

Site

MongoDB

A primeira coisa que precisamos é baixar, instalar e configurar o *MongoDB*, para isso Vá em “Produtos” e depois em “*Community Edition*”.

Clique no botão para conferir o conteúdo.

ACESSE

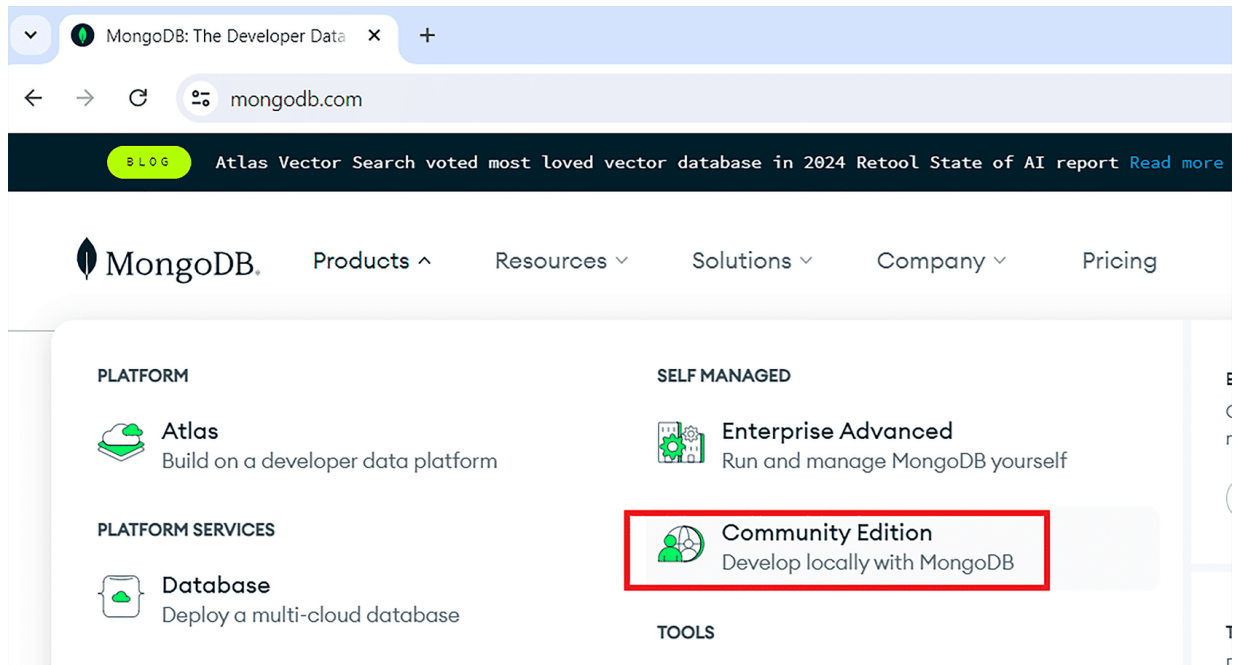


Figura 1 – Site do MongoDB

Fonte: Reprodução

#ParaTodosVerem: a Figura ilustrando o site do MongoDB. Contém fundo branco, letras em preto e azul, e contém também, dois retângulos vermelhos em “Produtos” e em “Community Edition”, mostrando onde devemos clicar para realizar o download do MongoDB. Fim da descrição.

Logo em seguida, você será direcionada para a página a seguir, e clique em *Download Community*.

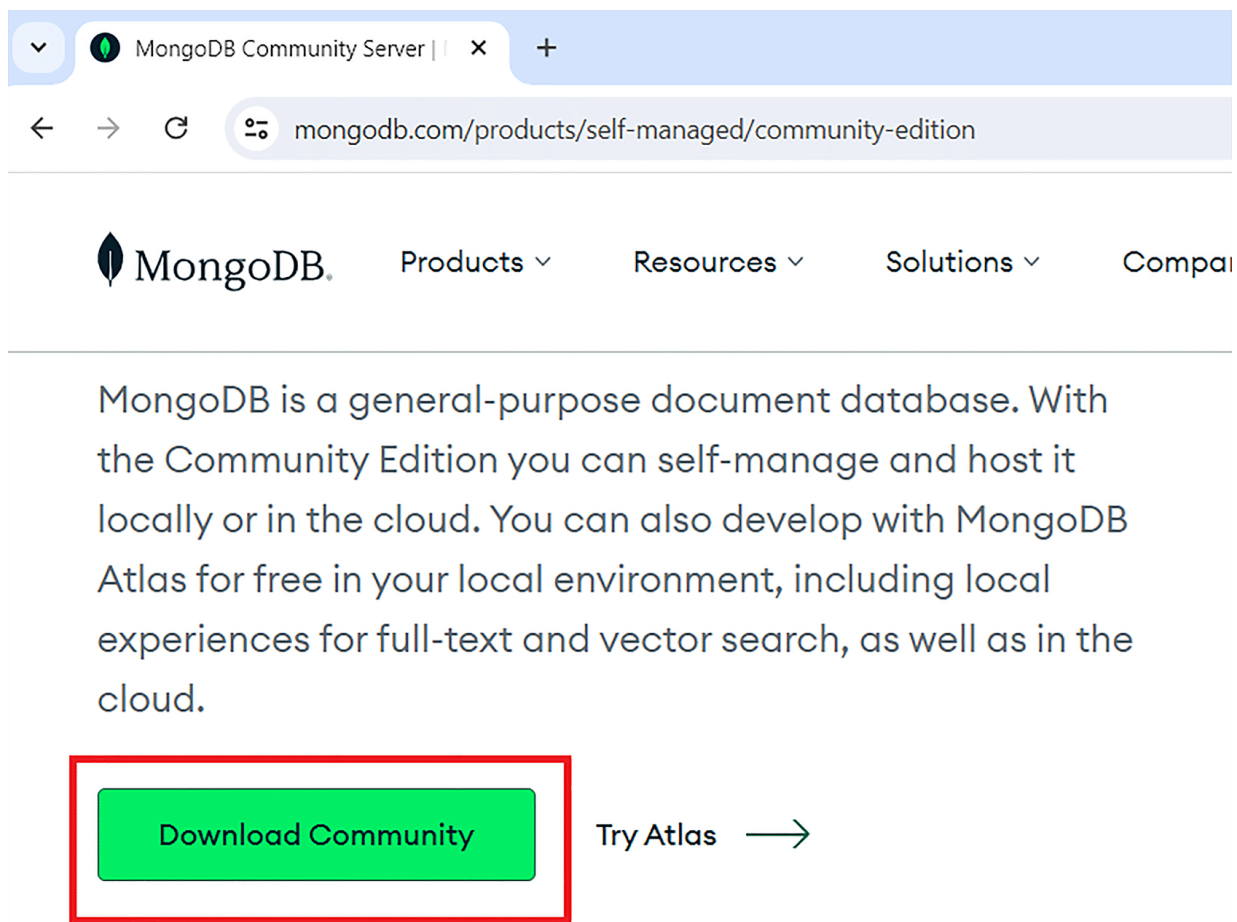


Figura 2 – *Download Community*

Fonte: Reprodução

#ParaTodosVerem: a Figura ilustrando onde devemos clicar para realizar o *download* do *MongoDB*. Contém fundo branco, letras em preto e azul, e contém também, um retângulo vermelho mostrando onde devemos clicar “*Download Community*” para realizar o *download* do *MongoDB*. Fim da descrição.

Ainda, você será direcionado para a página seguinte. Clique em *Select Package* e depois, escolha o seu sistema operacional. A instalação do *MongoDB* é padrão, basta clicar em *next* até o final. Após instalar, abra o *MongoDB*, você se deparar com a tela apresentada a seguir:

Quando você instala o *MongoDB*, é instalado também, o *MongoDB Compass* que é a parte gráfica do *MongoDB*. A parte gráfica permite uma maior facilidade na interação com o *MongoDB*. Entretanto, o foco desta Unidade de ensino é a apresentação dos principais comandos do *MongoDB*. Assim, iremos ainda, baixar o *Shell* do *MongoDB* para trabalhar somente com códigos. Entretanto, é aconselhável você copiar o *link* do endereço que aparece logo quando você abre o *compass*, o *link* é ilustrado dentro de um retângulo vermelho na Figura acima. Copie o *link* e feche o *Compass*.

Comandos *MongoDB*

Como mencionado anteriormente, *MongoDB* é um dos SGBDs mais utilizados no momento. Isso, porque o seu uso é totalmente fácil. A manipulação de dados, se dá através de comandos. Veremos os principais comandos a seguir. Para isso, vamos imaginar um cenário. Vamos imaginar que você deseja criar um banco de dados para uma concessionária de veículos. Para isso, é necessário criar um bando de dados utilizando o *MongoDB*.

A primeira coisa que devemos realizar é abrir o *Mongo Shell*.

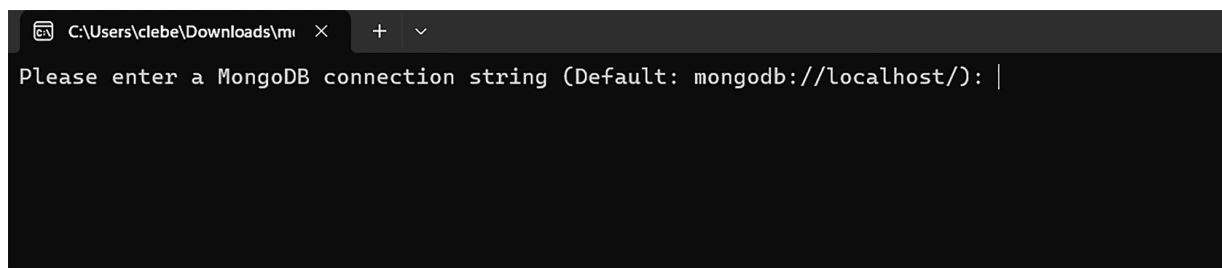


Figura 3 – MongoDB Shell

Fonte: Reprodução

#ParaTodosVerem: a Figura com o fundo preto e com as letras em branco, que ilustra o *MongoDB Shell*. Possui também, um cabeçalho com o diretório onde foi aberto o *MongoDB Shell*. Fim da descrição.

Agora, basta aperta a tecla <Enter> para realizar o *login* e entrar no *MongoDB Shell*.

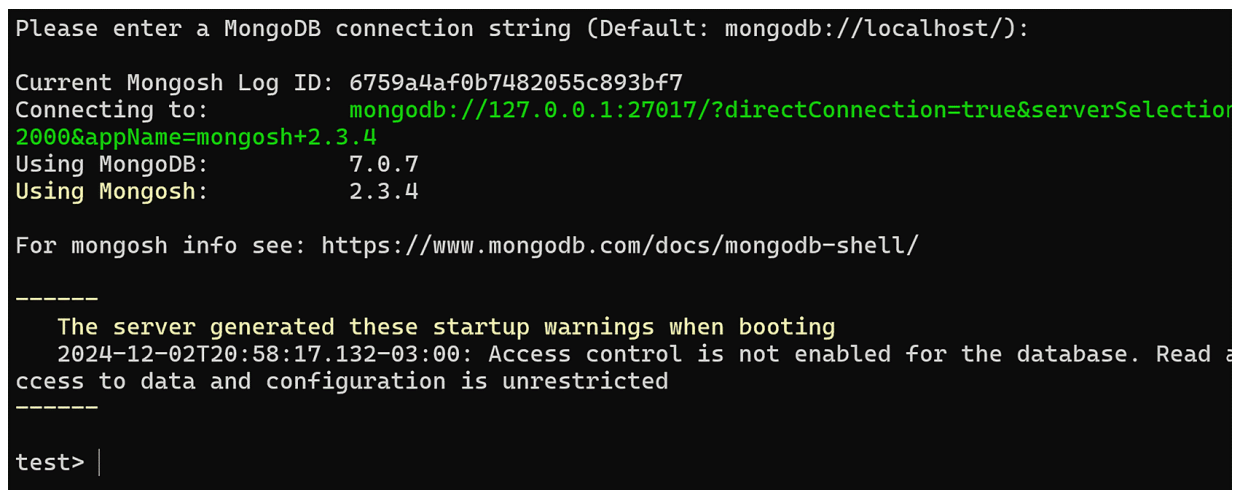


Figura 4 – Login no MongoDB Shell

Fonte: Reprodução

#ParaTodosVerem: a Figura com o fundo preto e com as letras em branco, verde e amarela, ilustra o *login* no *MongoDB Shell*. Possui também, um cabeçalho com o diretório onde foi aberto o *MongoDB Shell*. Fim da descrição.

Se você desejar, você poderá pressionar as teclas <ctrl> + <l> para limpar a tela. Depois de logado, devemos criar o banco de dados. A criação de banco de dados no *MongoDB* é extremamente fácil. Entretanto, antes de criar o banco, veremos quais são os bancos já existentes para isso, podemos dá o comando.

```
show dbs
```

Este comando listará todos os bancos já existentes.

Agora, podemos criar o nosso banco, para criar o nosso banco, usaremos o comando use, isso porque, para criar um banco basta apenas falar para o *MongoDB* que iremos utilizamos copie e cole no comando a seguir no *MongoDB Shell*.

```
use webCar
```

O nosso banco se chamará *webCar*, por isso use *webCar*. Agora, você irá perceber que o nome do banco aparece do lado esquerdo estará assim:

```
webcar >
```

Isso porque o banco de dados foi criado e o *MongoDB* já realizou a troca do banco. Assim, todo comando que executaremos agora, será executado dentro deste banco.

Agora, iremos realizar um teste. Se o nosso banco foi criado, e se executamos novamente o comando:

```
show dbs
```

O nosso banco deveria aparecer, correto? Faça o teste agora. Execute o comando *show dbs*.

Agora, você deve estar se perguntando, o que houve? Porque não apareceu o banco que eu acabei de criar?

O *MongoDB* é um SGBD muito inteligente, como o banco só foi criado, e não contém nenhum dado, nenhuma *collection* (estruturas para armazenar dados), ele entende que não é necessário mostrar para você, se não tem dados ainda, não é importante mostrar para você.

Agora, vamos criar uma *collection*. *Collection* são estruturas para armazenar dados. Se a gente fosse fazer uma comparação, uma *collection* seria semelhante a uma tabela no SQL. Para criar uma *collection*, usaremos o comando *createCollection()* copie o código a abaixo e cole no *shell*.

```
db.createCollection("Carro")
```

Todo comando que iremos executar, será inicia com *db*. Isso é necessário para que não seja necessário digitar novamente o nome do banco. Colocando apenas *db*. E o comando que queremos executar, o *MongoDB* irá entender que este comando será executado no banco em questão.

Agora, para testar, execute novamente o comando *show dbs* e você verá que o banco *webCar* será listado como banco criado.

Agora vamos inserir dados nesta *collections*. A inserção de dados pode ser feita por dois métodos. Veremos o primeiro. O *insertOne()* permite inserir um documento por vez na *collection*. Fazendo uma comparação, um documento seria semelhante a uma linha de uma tabela no SQL. Execute o comando a seguir:

```
db.Carro.insertOne({marca: "Fiat", modelo: "toro",  
valor:78.251, potencia:1.6})
```

O comando acima, insere um documento na *collection* Carro. Assim, para inserir um documento, é necessário digitar o *db* para especificar o banco, a *collection* que será inserir o documento e a função responsável por realizar a inserção. Neste caso, *insertOne*.

Há outra forma de inserir dados em uma *collection*. Agora vamos inserir dados, usando a função *insertMany()* é possível inserir vários documentos de uma vez só. Execute o comando a seguir:

```
db.Carro.insertMany([  
{marca: "volkswagen", modelo: "gol", valor:63.985, potencia:1.0},  
{marca: "fiat", modelo: "palio", valor:75.962, potencia:1.4},  
{marca: "fiat", modelo: "argo", valor:83.698, potencia:1.8},  
{marca: "chevrolet", modelo: "cruzes", valor:98.426, potencia:2.0},  
{marca: "peugeot", modelo: "207", valor:7.439, potencia:1.3},  
{marca: "chevrolet", modelo: "corsa", valor:18.742, potencia:1.2},  
{marca: "chevrolet", modelo: "tracker", valor:152.321, potencia:1.8},
```



```
{marca: "peugeot", modelo: "2008", valor:35.741, potencia:1.6},  
{marca: "volkswagen", modelo: "golf", valor:93.985, potencia:1.0},  
{marca: "chevrolet", modelo: "onix", valor:52.571, potencia:1.8},  
{marca: "honda", modelo: "HR-v", valor:169.985, potencia:2.0},  
{marca: "honda", modelo: "civic", valor:257.524, potencia:1.6},  
])
```

Agora, vamos consultar os documentos desta *collections*. Para consultar dados iremos utilizar a função *find()*. Execute o código abaixo:

```
db.carro.find()
```

No comando acima, estamos realizando uma consulta semelhante ao *select* do SQL. Este comando retornará todos os dados que estão nesta *collection*.

Agora, vamos supor que você queira consultar somente os carros da *Honda*. Se fosse no SQL, você usaria uma clausula *Where*, correto? No *find()* também podemos implementar alguns tipos de filtro. Execute o código abaixo:

```
db.carro.find({marca:"honda"})
```

Agora, será listado somente os carros da *Honda*.

Outro comando muito utilizado no *MongoDB*, é o *count()*. Este comando permite contar quantos documentos aquela *collections* tem. Execute comando abaixo:

```
db.carro.find({marca:"honda"}).count()  
db.carro.find().count()
```

Ao executar o primeiro comando será exibido a quantidade de carros da *Honda* que existe na *collection* carro. Já, ao executar o segundo código, será exibido a quantidade de carro no total que existem na *collection* carro.

Agora vamos supor que você queira alterar o modelo de um carro específico. Vamos supor que você queira alterar o modelo *HR-V* para *HR-V turing*. Para isso, você deverá utilizar o comando *update*. Execute o comando a seguir:

```
db.carro.updateOne({modelo:"HR-V"}, {$set:{modelo:"HR-V Turing"}})
```

Agora, execute o comando *find()* novamente para verificar se a alteração aconteceu.

Vamos supor que você ainda queira realizar algumas alterações. Vamos supor que você queira alterar o valor do modelo *HR-V Turing*. Na verdade, você quer apenas incrementar 100 reais no valor deste modelo. Para isso, você pode usar a função *inc*. Execute o código abaixo.

```
db.carro.update({modelo:"HR-V Turing"} , {$inc:{valor:100}})
```

Agora, novamente execute o comando *find()* para verificar se a alteração foi realizada.

Para deletar um documento, precisamos usar a função *deleteOne()*. Vamos excluir o modelo *HR-V Turing*. Para isso execute o comando abaixo:

```
db.carro.deleteOne( { modelo:"HR-V Turing" } )
```

Execute novamente o comando *find()* para verificar se o documento foi excluído.

Vamos supor agora, que você queira pesquisar os carros que estão acima de um determinado valor. Por exemplo, vamos supor que você queira saber quais são os carros que estão acima de R\$ 40.000,00 reais. Para isso você pode usar a função *find()* junto com *gt*. Execute o comando abaixo:

```
db.carro.find({valor: {$gt: 40000}})
```

Agora, você irá perceber que valores iguais e superior a R\$ 40.000,00.

Vamos pesquisar agora, por carros que tenham o valor menor que R\$ 40.000,00 para isso, você terá que usar a função *find()* novamente, só que agora com a ajuda do *lt*. Execute o comando abaixo:

```
db.carro.find({valor: {$lt: 40000}})
```

Muito provavelmente que agora, você esteja visualizando somente os carros que tenham valores abaixo de R\$ 40.000,00 a função *lt* é semelhante a função *gt* ela exclui o igual, para valores iguais e menores você deve usar o *lte*. Execute o código abaixo:

```
db.carro.find({valor: {$lt: 40000}})
```

A função *lt* é semelhante a função *gt*, ela exclui o igual, para pegar vamos inferiores e iguais, você também deve colocar o *e* no final. Execute o comando a seguir:

```
db.carro.find({valor: {$lte: 40000}})
```

A função *find()* é semelhante a função *select* no *select*. Ela garante que irá trazer todos os dados que estão na *collection*, mas não

garante a ordem. Para ordenar a consulta, devemos utilizar a função *sort*. Copie e execute o comando a seguir:

```
db.carro.find().sort({valor: 1})
```

O comando acima retorna os documentos ordenados por valores, isto é, retorna todos os carros ordenados por valor de forma crescente, isto é, do menor para maior, por isso o 1 no final. Para ordenar de forma decrescente, colocamos o -1. Execute o código a seguir:

```
db.carro.find().sort({valor: -1})
```

A função *sort()* também pode ordenar consultas com filtros. Copie e execute o comando abaixo:

```
db.carro.find({marca:"fiat"}).sort({valor: -1})
```

Como é possível observar, no *MongoDB* podemos combinar comandos. Ao executar o comando acima, será listado de forma decrescente todos os carros do modelo *Fiat*.

Outro comando muito utilizado e útil é o comando *distinct()*. Este comando possibilita distinguir os dados. Por exemplo, vamos supor que queremos saber se no nosso banco contém carros do modelo *Fiat*. Para isso, não precisamos dar um *find* e listar todos os carros da *Fiat*, ou até mesmo, todos os carros que estão no nosso banco. Execute o comando a seguir:

```
db.carro.distinct(marca)
```

Agora, será listada apenas as marcas que são diferentes. A sua consulta, não terá valores iguais.



Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

NoSQL e Big Data

Neste livro são abordados exemplos profundos de assuntos relacionados com bando de dados não relacional, tal como, *Big Data*.

GAUR, N.; DESHPANDE, A.; GOVINDAN, V. *NoSQL e Big Data*. Tradução de Andre Luis Tavares. 1. ed. São Paulo: Alta Books, 2019.

Seven Databases in Seven Weeks: A Guide to Modern Databases and the NoSQL Movement

Esta referência se trata de um excelente livro sobre NoSQL, aqui, o aluno pode realizar um estudo mais abrangente.

REDMOND, E.; WILSON, J. R. *Seven Databases in Seven Weeks: A Guide to Modern Databases and the NoSQL Movement*. Rio de Janeiro: Alta Books, 2012.

Entendendo Algoritmos: Um Guia Ilustrado para Programadores e Outros Curiosos

Esta referência se trata de um excelente livro sobre NoSQL, aqui, o aluno pode realizar um estudo mais abrangente.

KLEPPMANN, M. **Entendendo Algoritmos: Um Guia Ilustrado Para Programadores e Outros Curiosos**. Tradução de Carlos Szlak. 1. ed. São Paulo: Alta Books, 2017.

Big Data: Principles and Best Practices of Scalable Realtime Data Systems

Neste livro são abordados exemplos profundos de assuntos relacionados com bando de dados não relacional, tal como, *Big Data*.

MARZ, N.; Warren, J. *Big Data: Principles and best practices of scalable realtime data systems*. São Paulo: Alta Books, 2015.



Referências

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2019. (*e-book*)

HARRISON, G. *Next generation databases: NoSQL, NewSQL, and big data*. New York: Apress, 2015. 235 p.

HEUSER, C. A. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2011. (*e-book*)

MANNINO, M. V. **Projeto, desenvolvimento de aplicações e administração de banco de dados**. 3. ed. Porto Alegre: AMGH, 2014. (*e-book*)

MEDEIROS, L. F. **Banco de dados: princípios e prática**. São Paulo: InterSaberes, 2013. (*e-book*)

MOTA, F. S.; *et al.* **Modelagem digital**. Porto Alegre: SAGAH, 2019.

NEUKIRCHEN, A. *SQL Joins na prática!*, 2020 *Medium*. Disponível em: <<https://medium.com/@aneuk3/sql-joins-defcf817e8cf>>.

Acesso em: 20/12/2024.

OLIVEIRA, S. S. Banco de dados não-relacionais: um novo paradigma para armazenamento de dados em sistemas de ensino colaborativo. **Revista Eletrônica da Escola de Administração Pública do Amapá**, Macapá, v. 2 n. 1, p. 184–194, ago./dez. 2014.

Disponível em:

<<http://www2.unifap.br/oliveira/files/2016/02/35-124-1-PB.pdf>>. Acesso em: 14/10/2024.

PORTAL EDUCAÇÃO. **O que é tecnologia?** Portal Educação, 2022.

Disponível em: <<https://blog.portaleducacao.com.br/o-que-e-tecnologia/>>. Acesso em: 20/12/2024.

SANTOS, R. R. *et al.* **Fundamentos de Big Data**. Grupo A, 2021.

SILBERSCHATZ, A. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020. (*e-book*)

SILVA, L. F. C. *et al.* **Banco de Dados Não Relacional**. Grupo A, 2021.